WEEK 5

Create an application to demonstrate user-defined functions using Dart.

AIM: To create an application that demonstrate user-defined functions using dart.

THEORY:

User-Defined Functions: A user-defined function is a function created by the programmer to perform a specific task.

Syntax of a User-Defined Function:

```
returnType functionName(parameter1, parameter2, ... )
{
  // statements

  return value;
}
```

A function consists of:

1. Function Declaration (Definition) - It defines what the function does.
2. Function Call - A function is executed by calling its name.
3. Parameters - Parameters are variables declared in the function definition to receive values.
4. Arguments - are the actual values passed to a function during the function call.
5. Return Value - A function may return a value using the 'return' statement.

Types of User-Defined Functions:

Dart functions can be classified into four types:

1. Function without parameters and without return value
2. Function with parameters and without return value
3. Function without parameters and with return value
4. Function with parameters and with return value

Type 1: Function without Parameters and without Return Value

This function neither accepts parameters nor returns any value.

Example:

```
void greet()
{
   print("Welcome to Flutter!!");
}
void main()
{
   greet();
}
```

Output: Welcome to Flutter!!

Type 2: Function with Parameters and without Return Value

This function accepts parameters but does not return any value.

Example:

```
void display(String name)
{
   print("My Name is: $name");
}

void main()
{
   display("Alice");
}
```

Output: My Name is Alice

Type 3: Function without Parameters and with Return Value

This function does not accept parameters but returns a value.

Example:

```
int getNumber()
{
   return 100;
}

void main()
{
   int num = getNumber();
   print(num);
}
```

Output: 100

Type 4: Function with Parameters and with Return Value

This function accepts parameters and returns a value.

Example:

```
int add(int a, int b)
{
   return a + b;
}

void main()
{
   int sum = add(10,20);

   print(sum);
}
```

Output: **30**

PROGRAM1: [*functions_console.dart*] –

a) The following program demonstrates the types of user-defined functions. This program displays output on console.

```
void main()
{
  // 1. Without Parameters and Without Return Value
  welcome();

  // 2. With Parameters and Without Return Value
  displayName("Alice");

  // 3. Without Parameters and With Return Value
  int number = getNumber();
  print("Roll Number = $number");

  // 4. With Parameters and With Return Value
  int sum = add(10, 20);
  print("Sum = $sum");
}

//Function Declarations/Definitions:
// 1. Without Parameters and Without Return Value
void welcome() {
  print("Welcome to Dart");
}

// 2. With Parameters and Without Return Value
void displayName(String name) {
  print("My Name is: $name");
}

// 3. Without Parameters and With Return Value
int getNumber() {
  return 100;
}

// 4. With Parameters and With Return Value
int add(int a, int b) {
  return a + b;
}
```

OUTPUT:

Welcome to Dart

My Name is: Ajay

Roll Number = 100

Sum = 30

EXPLANATION for Program1:

Function Type	Function	Purpose
Without Parameters & Without Return Value	welcome()	Prints a welcome message.
With Parameters & Without Return Value	displayName(String name)	Accepts a name and prints it.
Without Parameters & With Return Value	getNumber()	Returns the value 100.
With Parameters & With Return Value	add(int a, int b)	Accepts two numbers and returns their sum.

There is no widget tree for the above program1, because it is a pure Dart program, not a Flutter UI program. It does not use any Flutter widgets such as MaterialApp, Scaffold, Text, or Button. It only executes functions and prints the output to the console.

Program Flow (instead of Widget Tree):

main()

- **welcome()**
  - print("Welcome to Dart")
- **displayName("Alice")**
  - print("My Name is: Alice")
- **getNumber()**
  - return 100
- print("Roll Number = 100")
- **add(10,20)**
  - return 30
- print("Sum = 30")

PROGRAM2: [*sum_app_console.dart*]

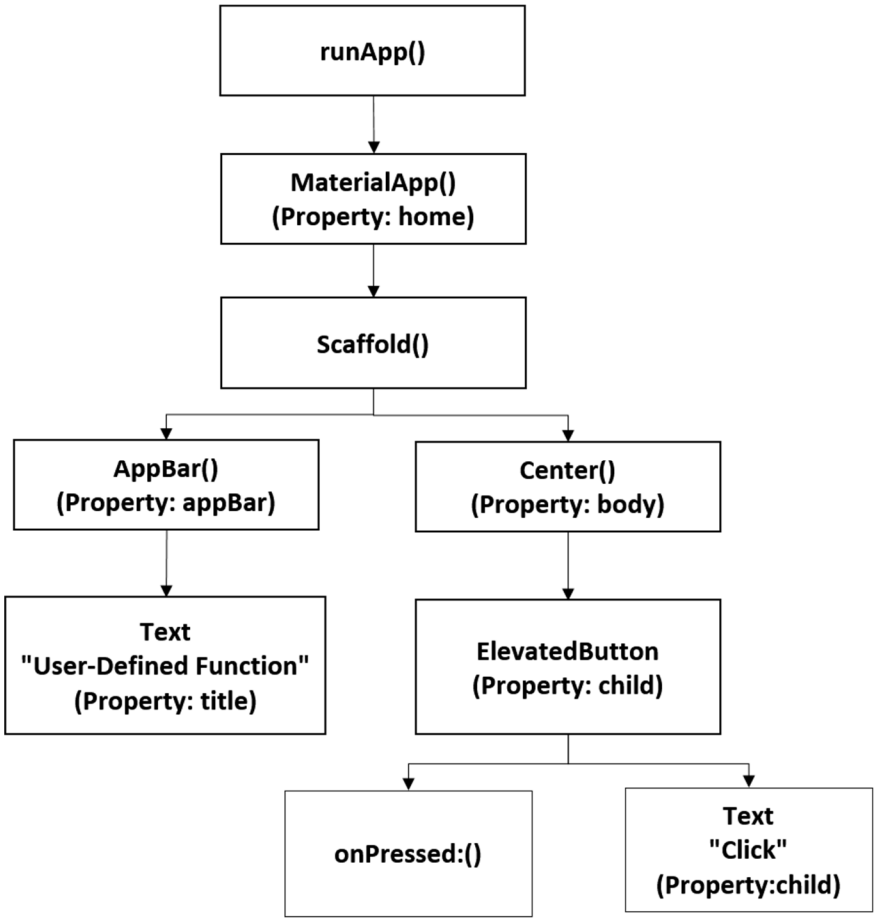
b) The following program prints sum of two numbers when a button is clicked in the flutter application. It displays output on debug console (not on application/output screen) when a button is clicked.

```
import 'package:flutter/material.dart';
void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text("User-Defined Function"),
        ), //AppBar
        body: Center(
          child: ElevatedButton(
            onPressed: () {       // Creates an anonymous function.
              int result = add(10, 20);
              print("Sum = $result"); // Output gets in console
            },
            child: const Text("Click"),
          ), //ElevatedButton
        ), //Center
      ),  // Scaffold
    ),  // MaterialApp
  ); //runApp
}

int add(int a, int b) {
  return a + b;
}
```

OUTPUT: Sum = 30

The Widget Tree for the above program is:



Program-2 Explanation:

Code	Explanation
`Import 'package:flutter/material.dart';`	Imports the **Material Design library**, which provides Flutter widgets like MaterialApp, Scaffold, AppBar, Text, Center, and ElevatedButton.
`void main()`	The main() function is the entry point of every Dart/Flutter application. Program execution starts from this function.
`runApp()`	Starts the Flutter application by initializing the Flutter framework and displaying the root widget.
`MaterialApp(`	The **root widget** of the application that provides the Material Design look and feel.
`home:`	Specifies the **first screen (home page)** of the application.
`Scaffold(`	Provides the **basic page structure**, including the AppBar and Body.
`appBar:`	Creates the **top application bar**.
`AppBar(`	A Material Design app bar displayed at the top of the screen.
`title:`	Specifies the **title displayed** in the AppBar.
`const Text("User-Defined Function")`	Displays the text "User-Defined Function" as the application title. const indicates that the text will not change.
`body:`	Defines the **main content area** of the application screen.
`Center(`	Aligns its child widget at the **center** of the screen.
`child:`	Specifies the widget that will be displayed inside the Center widget.
`ElevatedButton(`	Creates a **clickable Material Design button**.
`onPressed:`	A callback function that executes when the **button is pressed**.
`()`	Represents an **anonymous function with no parameters.** It runs only when the button is clicked.
`{ }`	Contains the statements to be executed when the button is pressed.
`int result = add(10, 20);`	Calls the user-defined function add(), passes the values 10 and 20, and stores the returned value in the variable result.
`print("Sum = $result");`	Displays the value of result in the Debug Console. Here, $result is an example of string interpolation.
`child: const Text("Click")`	Displays the text "Click" on the button.
`int add(int a, int b)`	Defines a user-defined function named add() that accepts two integer parameters.
`return a + b;`	Calculates the sum of a and b and returns the result to the calling function.

WEEK-5 VIVA QUESTION AND ANSWERS

1. What is Flutter?

Ans: Flutter is Google's open-source UI toolkit for building cross-platform applications.

---

2. Why do we use runApp()?

Answer: runApp() launches the Flutter application and displays the root widget on the screen.

---

3. What is MaterialApp?

Answer: MaterialApp is the **root widget** that provides the Material Design structure, theme, navigation, and application configuration.

---

4. What is the purpose of Scaffold?

Answer: Scaffold provides the **basic page layout**, including the AppBar, Body, Floating Action Button, Drawer, and Bottom Navigation Bar.

---

5. Why is AppBar used?

Answer: AppBar creates the **title bar displayed** at the top of the application.

---

6. Why is the Center widget used?

Answer: The Center widget aligns its child widget at the center of the screen.

---

7. What is an ElevatedButton?

Answer: ElevatedButton is a **Material Design button** that performs an action when pressed.

---

8. What is the purpose of the onPressed property?

Answer: onPressed is a callback function that specifies **the action to perform** when the button is clicked.

---

9. Why do we write () { } after onPressed?

Answer: () defines an anonymous function with no parameters. The code inside { } executes only when the button is pressed.

---

10. What is a callback function?

Answer: A callback function is a function that is passed to another function or widget and is executed later when a specific event occurs.

---

11. What is the purpose of the add() function?

Answer: add() is a user-defined function that accepts two integers as parameters and returns their sum.

---

12. What are the parameters in the add() function?

Answer: The parameters are int a and int b, which receive the values passed during the function call.

---

13. What is the return type of the add() function?

Answer: The return type is int because the function returns an integer value.

---

14. What is string interpolation?

Answer: String interpolation is the process of **inserting the value of a variable into a string using the $ symbol.**

Example:

```
print("Sum = $result");
```

---

15. Where is the output displayed in this program?

Answer: The output is displayed in the **Debug Console**, not on the application screen.

Output:

Sum = 30

---

16. What is the purpose of "const" before the Text widget?

Answer: const creates a compile-time constant object, improving performance by preventing unnecessary widget recreation when the widget's data never changes.

---

17. What will happen if "onPressed" is set to null?

Answer: The button becomes disabled and cannot be pressed.

Example:

```
ElevatedButton (
  onPressed: null,
  child: Text("Click"),
)
```

---

18. Why is print() used?

Ans: To display output in the Debug Console.